Beyond the Agent Object

Paragraph-by-paragraph exzerpt — keywords only

- **Mapping:** ¶n = blank-line source block in `paper/main.typ`; one keyword entry per prose block; original list items retained separately; equations, tables, process orders retained as compact notation

1 Abstract

- ¶4 **ABM convention:** agent object = identity + state + behavior; predefined types as construction unit; possible concealment of population-level interaction processes; **ECS comparison dimensions:** state composition + behavior organization + schedule semantics; component signatures → structure; queries → participants; dependencies → schedule + visibility; idiomatic agent-centered comparison; changing roles + shared mechanisms + order equivalence; Sugarscape reconstruction + reproduction + disease; cache/multicore/GPU as secondary engineering evidence; ECS = scientific specification architecture, not optimization only
- ¶5 **Document status:** manuscript scaffold; claims + required evidence + qualifications; finished prose only after implementation/analysis

2 Introduction

- ¶7 **Sugarscape:** heterogeneous vision + metabolism + endowment + lifespan; regenerating landscape → movement + harvest + wealth + starvation/old-age death; reproduction/disease → overlapping, changing roles; female/male + fertile/infertile + susceptible/infected/immune; persistent-state roles vs. derived roles; architectural chain = state composition → process selection + transformation → scheduling; data layout/execution = secondary consequence
- ¶8 **ABM explanation:** heterogeneous agents + interaction → macro regularities; common mapping = scientific agent → software object combining identity/state/behavior; intuitive but non-neutral → agent taxonomy + agent-local steps + population processes as repeated invocations
- ¶9 **ECS starting point:** entity → identity; component → state/capability; system → transformation over required signature; composition + processes, not class membership + local methods
- ¶10 **Research aim:** usefulness of ECS architectural change for scientific ABMs; manifestation in concrete modeling choices/practices

2.1 Why architecture matters

- ¶12 **Scientific agents:** person/household/firm/bank/government/citizen; individual states + actions + interaction-driven evolution; architecture → scientific description translated into executable state/processes
- ¶13 **Heterogeneity:** ABM strength; common encodings = parametric values or predefined categories; third form = structural heterogeneity; agents differ in possessed state variables/capabilities independently of fixed hierarchy; ECS support through component composition
- ¶14 **Interactions:** relational, multi-agent → population dynamics + nonlinear feedback; one-agent method execution → process ownership, participation, update timing obscured; simultaneity → observation + intention + conflict resolution + update phases
- ¶15 **Parallel agent-centered feasibility:** FLAME GPU; same type/state → concurrent agent functions; messages → safe interaction; layers/dependency graphs → order; parallelism possible under explicit constrained model = types + states + messages + dependencies
- ¶16 **ECS alternative:** heterogeneous population + homogeneous operations on shared components; component storage → batching; dependencies → safe scheduling/parallel basis; no automatic parallelism; execution unit = population process, not individual method

2.2 What changes with ECS?

- ¶18 **Comparison structure:** three analytically separate dimensions; separate claims; architectural force from connection
- ¶19 **State composition:** question = agent-constituting state/capabilities; ECS = current component signature defines schema
- ¶20 **Behavior organization:** question = location of shared transition laws; ECS = systems transform query-selected populations

- ¶21 **Schedule semantics:** question = visible state + update time + equivalent orders; ECS = dependencies/phases specify order + visibility + admissible reordering
- ¶22 **Dimension relation:** composition + behavior = executable description; schedule = implemented transition; no collapse; common ECS interface = role requirements → selected population → dependency-constrained observation and modification; storage performance = secondary consequence
- ¶23 **Investigation:** three research questions
- ¶24a **RQ1:** ECS effects on overlapping/changing role representation and modification vs. idiomatic agent-centered same-model implementation
- ¶24b **RQ2:** system organization effects on locality + reuse + substitutability + inspectability vs. agent-centered activation
- ¶24c **RQ3:** dependencies/phases → visibility + timing; declared conditions for outcome-preserving alternative orders
- ¶25 **Contribution:** conceptual separation of three dimensions; parametric/type/compositional heterogeneity distinction; ABM ↔ ECS concept map; same classical economic ABM in agent-centered + ECS forms; overlapping + changing capabilities; staged simultaneity from dependencies; controlled architecture/schedule treatments; separate cache/multicore/GPU evaluation

3 Conceptual foundations

- ¶27 **Architecture-neutral concepts:** population heterogeneity forms; world states + transitions + schedules + trajectories + model time; no prior behavior-location or storage assumption

3.1 Three kinds of heterogeneity

- ¶29 **Dimensions:** parametric + type-based + compositional; distinct, nonexclusive, combinable; complete admissible states $\mathcal{W}$; current finite nonempty agents $I(W)$; size $N(W)$; entry/exit/replacement allowed; no fixed population/IDs/clock; Sugarscape = traits + sex/infection roles + demographic turnover
- ¶30 **Complete state:** agent descriptors + environment + remaining model state + seeded RNG state; complete state + fixed schedule → determinate successor
- ¶31 **Agent state:** admissible space $X_{W(i)}$; current value $x_{W(i)} \in X_{W(i)}$; world-indexed schema → within-run structural change + alternative same-time schemas; dynamic variables + stable traits
- ¶32 **Parametric heterogeneity:** projection $p_X : X \to \Theta_X$ onto scientifically stable-trait coordinates; same schema group + unequal projected values; dynamic position/memory differences = state heterogeneity, not parametric; Sugarscape vision/metabolism example
- ¶33 **Type-based heterogeneity:** exhaustive nominal map $\tau_W : I(W) \to \mathcal{T}$; unequal assignments → disjoint type partition; type may determine schema/law; within-type state/signature variation possible; female/male class example vs. exclusive zero-sized tags; scientific nominal classification ≠ language inheritance
- ¶34 **Compositional heterogeneity:** agents differ in component sets; universe $\mathcal{U}$; scientifically relevant types $c_1, ..., c_m$; complete vs. structural signature
- ¶35 $\kappa_W : I(W) \to \mathcal{P}(\mathcal{U}); \quad \kappa_W^{\text{str}}(i) = \kappa_{W(i)} \cap \{c_1, ..., c_m\}$
- ¶36 **Signature distinction:** full implementation signature κ_W vs. analytical structural projection; mandatory components insufficient for heterogeneity; schedule-only buffers retained in full signature, excluded from scientific list absent substantive interpretation
- ¶37 **Component at definition level:** presence tag only; no assigned data domain/parameter block; compositional definition independent of later ECS storage representation
- ¶38 **Incidence indicator:** relevant component presence by agent/component
- ¶39 $\chi_{ik}(W) = 1$ iff $c_k \in \kappa_W^{\text{str}}(i)$; otherwise 0
- ¶40 **Incidence vector:** indicators collected per agent
- ¶41 $\chi_{i(W)} = (\chi_{i1}(W), ..., \chi_{im}(W)) \in \{0,1\}^m$
- ¶42 **Incidence matrix:** ordered agents × relevant components; $\boldsymbol{C}(W) \in \{0,1\}^{N(W) \times m}$; empirical structural-signature distribution
- ¶43 $\pi_{W(z)} = N(W)^{-1} \left| \left\{ i \in I(W) \texttt{ mid } \chi_{i(W)} = z \right\} \right|$
- ¶44 **Compositional heterogeneity iff:** two unequal structural signatures; equivalently, more than one positive-mass incidence vector
- ¶45 **Joint architecture descriptor:** nominal type + component signature + admissible state space + current state
- ¶46 $a_{W(i)} = \left(\tau_{W(i)}, \kappa_{W(i)}, X_{W(i)}, x_{W(i)}\right); \quad x_{W(i)} \in X_{W(i)}$

- ¶47 **Non-exhaustiveness:** equal type + signature + parameters still compatible with unequal dynamic states
- ¶48 **Dynamic composition:** persistent agent's structural signature changes, or entry/exit/replacement changes π_W; heterogeneity not ECS invention; ECS contribution = explicit incidence/change without exhaustive combination classes

3.2 Transitions, schedules, trajectories, and time

- ¶50 **Time-model neutrality:** no assumed ticks/synchrony; admissible relation $\rightarrow \subseteq \mathcal{W} \times \mathcal{W}$; schedule Σ selects + composes activations/systems $\rightarrow T_\Sigma$ with admissible successor
- ¶51 **Simulation run:** trajectory
- ¶52 $\omega = (W_0, W_1, ...); \quad W_{n+1} = T_{\Sigma_n}(W_n)$
- ¶53 **Index vs. time:** n = transition index; clock maps world state into ordered time domain; $t_n = \text{clock}(W_n)$; shorthand only after fixed trajectory
- ¶54 $I_n = I(W_n); \quad x_{n(i)} = x_{W_n}(i); \quad \kappa_{n(i)} = \kappa_{W_n}(i); \quad \pi_n = \pi_{W_n}$
- ¶55 **Discrete case:** one full schedule → one tick → $t = n$; event case → irregular t_n, possibly multiple transitions at same clock time
- ¶56 **Ordered phases:** within complete transition; phase-indexed states
- ¶57 $W_{n,0} = W_n; \quad W_{n,P} = W_{n+1,0} = W_{n+1}$
- ¶58 **Phase meaning:** visibility/update boundary, not elapsed time; same-clock phases possible; micro-index only for scientific mechanism; common definitions support sequential/simultaneous/phased/event models
- ¶59 **Three distinctions:** simultaneity = common information + no observed committed peer decision; concurrency = independently schedulable interleaving without process change; parallelism = simultaneous hardware execution; simultaneity can execute serially; sequential model can parallelize within activation
- ¶60 **Simultaneous phase:** phase-entry state frozen; each assigned process forms proposal
- ¶61 $\delta_{k,n,p} = P_{k(W_{n,p})}, \quad k \in K_{n,p}$
- ¶62 **Joint resolution:** proposals + phase-entry state → next visible state
- ¶63 $W_{n,p+1} = \text{resolve}_{n,p}\left(W_{n,p}, \left(\delta_{k,n,p}\right)_{k \in K_{n,p}}\right)$
- ¶64 **Sequential phase:** predecessor-produced intermediate state visible; common simultaneous staging sequence
- ¶65 `observe → form intentions → resolve conflicts → apply actions → learn`
- ¶66 **Qualification:** not all stages always needed; scientifically sequential ≠ forced simultaneity for convenience; phases specify observation + effect visibility, not software architecture

4 Positioning in the literature

4.1 ABM tools and agent-centered design

- ¶83 **Toolkit variation:** three dimensions implemented differently; individual kind + fields generally more explicit than independently composed signature; native τ, X, x; scientific κ emulated through classes + interfaces + flags + traits + selectors; behavior/schedule via methods, external functions, population operations, layered kernels
- ¶84 **Mesa:** Python `Agent` subclass + instance attributes + methods; model-level `AgentSet` with `do`/`shuffle_do`; filtered/grouped/fixed/random traversal but object entry; classes/attributes → natural τ, X, open schema; mixins/delegates/dynamic fields/capability fields/selectors → overlapping κ + staged/process-oriented passes
- ¶85 **MASON:** Java classes/interfaces + `Steppable`; fields → schema; scheduler → `step(SimState)`; helpers/environment/coordinators also steppable; priority queue → async events + ordered phases + fixed/random sets; interfaces/delegation/composition/flags → overlapping capabilities without subclass combinations; interface incidence resembles κ, but not ECS storage-query basis
- ¶86 **NetLogo:** turtle/patch/link + breed partitions + breed/common owned variables → τ, X, x; procedures not class methods, but `ask` in selected agent context + randomized serial changes; multiple passes/temporary state → decision/application separation; runtime breed change → lifetime schema change; variables/lists/links/predicates/agentsets → overlapping capabilities; breeds remain exclusive nominal categories
- ¶87 **Agents.jl:** concrete structs; `Union` or closed `@multiagent` variants; type/variant → τ, fields → X, multiple dispatch; external `agent_step!` + `model_step!`; scheduler selects activation; custom schedules +

buffers + repeated passes → shared/simultaneous processes; `EventQueueABM` → continuous time; traits/ delegation/flags/predicates approximate κ; built-in heterogeneous storage remains closed-type/variant centered

- ¶88 **FLAME GPU:** fixed variables by agent type; functions by agent state + layer; GPU population kernels; messages → communication phases; layers/dependency graph → order; explicitly phased/ concurrent activation; eligibility/storage still agent type/state rather than independent components
- ¶89 **Survey inference:** not simple agent-centered/ECS binary; substantial behavior/schedule variation; heterogeneous state usually anchored in kind + record + breed + variant; ECS changes anchor to κ + signature selection + structural change; same signature links state schema to process participation; storage = separate question

4.2 ECS, data-oriented design, and concurrency

- ¶98a **Required literature:** ECS origins; composition over inheritance
- ¶98b **Required literature:** formal ECS semantics; deterministic concurrency
- ¶98c **Required distinction:** architectural pattern vs. archetype storage

4.3 ECS in ABM and multi-agent systems

- ¶100a **Required literature:** ECS-based ABM engines; feasibility + CPU-parallel performance
- ¶100b **Required literature:** HECATE; ECS ↔ multi-agent engineering
- ¶100c **Required literature:** GPU ABM; agent functions/layers/messages/ kernels resembling population systems

4.4 Proposed gap

- ¶102 **Novelty claim status:** intended
- ¶103 **Prior work:** ECS-ABM feasibility + performance potential; **paper focus:** ECS as scientific architecture → dynamic component composition + structural heterogeneity; population-level systems; simultaneous interaction formulation
- ¶104 **Literature work pending:** reproducible search across ABM + IBM + MAS + component/ process-oriented simulation + FLAME/GPU + data-oriented science; no "first"/"unexplored" before documented search; add ODD + activation regimes + component ABM + GPU conflict resolution

5 Agent-centered baseline

5.1 Agent object convention

- ¶107 **Common mapping:** scientific agent → software object; identity + named type/record; external state + internal beliefs/preferences/memory; behavior via inheritance/dispatch/calls; broad architectural "object," no required OOP language or inheritance
- ¶108 **Execution:** tick/event scheduler → selected agent → step/handler; inspect self + neighbors/shared state → choose + mutate self/other/environment; repeated local invocations → population transition; toolkit order variants = fixed/random + buffers + event queues + types + custom schedules
- ¶109 **Principal unit:** object for description + execution; record/type → state; "what does agent do?" → behavior; one participant method → interaction entry; architectural commitment, not ABM necessity; alternative = identity- separated state + condition-selected population transformations
- ¶110 **Paper definition:** agent-centered = complete representation as primary individual schema + representation activation as primary behavior entry; schedule semantics + physical storage independent

5.2 Behavior located in agents

- ¶68 **Agent-level behavior:** heterogeneity describes differences, not behavior location; descriptor $a_{W(i)}$; available information $N_{i(W)}$; activation → agent transition selected by type/signature/state/world
- ¶69 $F_{i(W)} = f_{\tau_{W(i)}}\left(\kappa_{W(i)}, x_{W(i)}, N_{i(W)}, W\right)$
- ¶70 **Operator meaning:** returns world state; type → dispatch; signature → branch/delegated capabilities; state → dynamic + parameter inputs; world → remaining state; may mutate self/ others/environment; may replace signature + schema + state; agent-centered architecture permits compositional/dynamic heterogeneity; defining feature = transformation reached through specific agent activation
- ¶71 **Scheduler:** individual operators → population transition; activation order σ_W
- ¶72 $W^0 = W; \quad W^r = F_{\sigma_{W(r)}}(W^{r-1}); \quad T_{\sigma_W}(W) = W^{N(W)}$

- ¶73 **Sequential visibility:** activation r observes predecessor state; earlier changes visible later; order relevant under noncommuting operators
- ¶74 $F_i \circ F_j \neq F_j \circ F_i$
- ¶75 **Single-step timing opacity:** perception → choice → action → resolution → learning inside one invocation despite distinct scientific information/update times; read-write sequence exposes intermediate states absent explicit buffers or phases
- ¶76 **Behavior/schedule separation:** agent activation may emit intention, not immediate mutation; simultaneous proposal evaluation
- ¶77 $p_i = G_{i(W)}$
- ¶78 **Model-level resolution:** collection of proposals jointly applied; Sugarscape same citizen state + two schedules: shuffled sequential = immediate citizen move/harvest; synchronous = all proposals from frozen occupancy + landscape, then contested-cell resolution; isolates information semantics from intention-code location

5.3 From agent types to state schemas

- ¶80 **Common framework default:** declared kind → admissible fields; schema map from types to state spaces; several types may share schema; some spaces unused; non-necessary mapping because dynamic fields/delegation/flags/ traits/model tables can make effective schema exceed nominal type
- ¶81 **Fixed type/schema lifetime:** frequent agent-centered default, not universal property

5.4 Strengths and investigated limitations

- ¶91a **Strength:** ordinary-language autonomous-individual alignment
- ¶91b **Strength:** complete one-agent state/behavior inspection in one place
- ¶91c **Strength:** individualized cognition/event logic natural
- ¶91d **Strength:** mature ABM framework tooling
- ¶93a **Investigate, not assume:** overlapping capabilities → hierarchy/ conditional growth
- ¶93b **Investigate, not assume:** shared process across types → duplication
- ¶93c **Investigate, not assume:** simultaneity → observation/mutation separation difficulty
- ¶94 **Comparison requirement:** idiomatic reference, not weak inheritance strawman; Julia ≠ conventional class OOP; "agent-centered" terminology; stronger OOP claim only with representative class implementation or narrowed language

6 Entity Component Systems as a modeling architecture

- ¶114 **Chapter shift:** engine/MAS feasibility → scientific model construction consequences
- ¶115 **Architecture chain:** component composition + transition-law location → queries/effects → process schedule + update visibility; same information may support storage/batching; storage ≠ scientific semantics; memory layout ≠ schedule; explicit required-state account from composition → participation → dependency/schedule analysis; execution-loop effects evaluated separately

6.1 Entities, components, systems, resources

- ¶117a **Entity:** identifier; no intrinsic data/behavior
- ¶117b **Component:** focused entity-associated state unit
- ¶117c **Signature:** entity's current component set
- ¶117d **Query:** state-dependent signature/predicate selection
- ¶117e **System:** transformation over query-selected components/entities
- ¶117f **Resource:** model-level parameters/RNG/indexes/statistics
- ¶117g **Structural change:** entity/component addition or removal
- ¶118 **ECS schema binding:** signature rather than nominal type; component domain map Φ; entity state space = product of present-component domains
- ¶119 $X_{W(i)} = \prod_{c \in \kappa_{W(i)}} \Phi(c)$
- ¶120 **Dynamic/stable separation:** $X_{W(i)} \equiv S_{W(i)} \times \Theta_{W(i)}$ when separable; parameter projection identifies comparison coordinates; mutable/replacement-varying parameter remains complete simulation state and behavioral-law parameter

6.1.1 Queries

- ¶122 **Query = process role:** evaluated against current world, not fixed lifetime subset; uses current $I(W) + \kappa_W$

- ¶123 **Query representation:** required set + excluded set + optional value predicate
- ¶124 $E_{Q(W)} = \left\{ i \in I(W) \texttt{ mid } \mathcal{C}_Q^+ \subseteq \kappa_{W(i)}, \mathcal{C}_Q^- \cap \kappa_{W(i)} = \varnothing, \psi_{Q(i,W)} = 1 \right\}$
- ¶125 **Specializations:** ordinary required-signature query; value-filtered population; excluded capability, e.g. positioned but immobile; persistent entity + changed composition/value → changed membership
- ¶126 **System input:** finite query family
- ¶127 $\boldsymbol{Q}_k = \left(Q_{k1}, ..., Q_{km_k} \right)$
- ¶128 **Population vector:** current result per query; $m_k \geq 0$
- ¶129 $\boldsymbol{E}_{k(W)} = \left(E_{k1}(W), ..., E_{km_k}(W) \right)$
- ¶130 **Empty family:** model-level-only process; multiple queries = distinct participant roles or whole populations for joint match/aggregate/process; broader than independent invocation; Core ECS principle retained = declared query inputs

6.1.2 Resources

- ¶132 **Resource:** whole-model rather than one-entity state; resource-label set $\mathcal{G}$ + domain map Ψ; resource product in world state
- ¶133 $r_W \in \prod_{g \in \mathcal{G}} \Psi(g)$
- ¶134 **Examples:** parameters + seeded RNG + occupancy + traces + replacement queues + aggregates; read-only or evolving; stochastic behavior = transition conditional on RNG resource state
- ¶135 **Dependency universe:** tagged component/resource union $\mathcal{A}$; declared reads/writes = conservative type-level summaries of every possibly accessed kind, despite entity-specific actual access

6.1.3 Systems

- ¶137 **System specification:** query family + reads + writes + function
- ¶138 $\mathcal{S}_k = (\boldsymbol{Q}_k, \text{Read}_k, \text{Write}_k, F_k)$
- ¶139 **Evaluation:** read projection $\rho_{k(W)}$ + selected entities → new declared outputs; induced world transition
- ¶140 $T_k : \mathcal{W} \to \mathcal{W}$
- ¶141 **Frame condition:** outside Write_k unchanged
- ¶142 **Declaration completeness:** query-inspected presence/absence = read; component addition/removal = write; covers value access + membership change
- ¶143 **System effects:** component/resource updates + entity/component creation/removal; schedule → query observation + update visibility + system composition; deferred structural commit = engine constraint, not system definition
- ¶144 **Distinct roles:** optional match relation
- ¶145 $\mathcal{M}_{k(W)} \subseteq \prod_{j=1}^{m_k} E_{kj}(W)$
- ¶146 **Matching:** self-exclusion + spatial/institutional eligibility; avoids full Cartesian interaction; tuple kernel → indexed proposal
- ¶147 $d_{k,\boldsymbol{i}}(W) = f_{k\left(\boldsymbol{i}, \rho_{k(W)}\right)}$
- ¶148 **Simultaneous tuple semantics:** all kernels observe same W; joint proposal resolution → system transition
- ¶149 $T_{k(W)} = \text{resolve}_{k\left(W, \boldsymbol{d}_{k(W)}\right)}$
- ¶150 **Collection-wise system:** complete population vector → internal matching/aggregation/sampling/arbitration; natural market clearing + path conflict + survivor-pool replacement; unequal intermediate observation times → internal schedule or scientifically meaningful ordered systems
- ¶151 **Connection:** semantics + dependency analysis + testing + possible parallelism; unaffected reads/writes → concurrency candidate; contention/input change → ordering + reduction + arbitration + phase

6.2 Compositional heterogeneity

- ¶153 **Operationalization:** signature satisfies query → participation; independently overlapping populations; firm = exporter + borrower; citizen = female + infected + predicate-fertile; no combination-specific type
- ¶154 **Incidence duality:** schema + process eligibility; new signature need not require implementation because each system selects needed components; capability conjunction still explicit in query/rule
- ¶155 **Structural change:** schema + membership change; scientific timing = schedule choice; staged phase-boundary commit → traversal safety; ECS exposes change but does not choose visibility time

6.2.1 Component-incidence example

- ¶157 **Citizens:** A = Position + Female + Infection; B = Position + Female; C = Position + Male + Infection; D = Position + Male
- ¶158 **Interpretation:** mandatory Position → no observed heterogeneity; Female/Male mutually exclusive; Infection overlaps sex; infection system → A, C; reproduction → sex tags then age/wealth/ neighborhood predicates; recovery removes Infection while preserving identity/unrelated state → lifetime composition + membership change; fertility derived, not stored component

6.3 Thinking in systems rather than agents

- ¶160 **Question shift:** "agent action during step?" → "model transformation + entities with required state?"; executable unit = complete representation → population process
- ¶161 **Explicit processes:** perception + choice + movement + matching + learning + entry + exit, each with participants/visibility; no reconstruction from agent-step fragments; decomposition forces one-vs.-ordered-process and visibility decisions; scientific process ↔ system not automatically 1:1; multi-system market clearing or combined maintenance possible; mapping becomes explicit choice
- ¶162 **Sugarscape process organization:** growback → landscape; movement → positioned/vision/ wealth citizens; resolution → all destinations; infection → infected only; lifecycle → metabolism/age; reproduction → eligible female/ male match; each citizen in multiple processes; period advanced by process sequence, not one citizen invocation
- ¶163 **Substitution boundary:** alternative movement/transmission/ inheritance/matching F_k with same query/state contract → unchanged entities + storage + unrelated systems; new inputs/outputs → declarations/dependencies change; system separation ≠ incompatible-rule interchangeability; localized competing mechanism under common initialization/context/outcomes → controlled comparison, not new agent taxonomy
- ¶164 **Inspectable coupling:** read of output → possible order; overlapping writes → sequencing/ reduction/arbitration; disjoint outputs + shared read-only input → independence candidate; testing = constructed world + allowed outputs + phase invariants; conservative access sets ≠ scientific correctness proof; accesses visible, substantive equation/information/schedule justification not
- ¶165 **Agency retained:** agent identity + private information + goals + dispositions + memory + feasible actions + consequences; components hold entity-specific quantities; systems hold shared laws; autonomy = modeled information/decision structure, not method ownership; individualized policy components possible; whole-agent behavior inspection potentially harder
- ¶166 **Architectural gain:** component = individual state + system eligibility; reads/writes extend contract into schedule; capability → using process → observed state → effect visibility without translation among type hierarchy + activation routine + separate dependency account

6.4 Interaction phases and concurrency

- ¶168 **Executable phase semantics:** queries + proposal state + access sets + resolution systems; proposal ≠ elapsed instant/immediate visibility; temporal meaning from query phase + commit boundary
- ¶169 **Snapshot and resolution:** immutable phase-entry state + isolated proposal writes → concurrent proposal candidate; conflicting effects → resolver selects + aggregates + commits; stable resolution = model rule, not mere sync; synchronous Sugarscape proposals fixed before contest; computation order gives no advantage
- ¶170 **Conservative independence test:** no cross-system write/read or write/write conflicts
- ¶171 $\mathrm{Write}_j \cap (\mathrm{Read}_k \cup \mathrm{Write}_k) = \varnothing$ and $\mathrm{Write}_k \cap \left(\mathrm{Read}_j \cup \mathrm{Write}_j\right) = \varnothing$
- ¶172 **Implication:** complete declarations → neither changes other's input/output → commuting transitions + order-invariant successor; shared reads harmless; component/resource granularity → sufficient, not necessary; disjoint entity subsets or commutative reductions may allow overlap with precise partition/reduction/arbitration, never uncontrolled shared write
- ¶173 **Determinism conditions:** proposal/resolution independent of runtime schedule; shared mutable RNG = write dependency + shock reassignment risk; draws partitioned/indexed by replicate + phase + entity + event; floating reductions need fixed partitions + tie-breaking + order; deterministic concurrency derived from effect restrictions, not ECS label
- ¶174 **Concurrency limits:** shared-state contention → order/buffer/reduce/ arbitrate; barriers → costs; speedup depends on independent work/cost ratio; dependency graph preserves declared semantics only; ECS cannot decide simultaneity + conflict grouping + learning observation; scientific commitments

6.5 Data-oriented engineering consequences

- ¶176 **Logical vs. physical:** composition ≠ storage; map-per-entity ECS possible; structure-of-arrays agent records possible; representation alone ≠ performance; opportunity only when logical signatures organize storage + execution
- ¶177 **AoS:** full records contiguous; narrow position/speed sweep loads unused fields/cache data; **component/archetype layout:** same components or same- signature tables with column arrays; query → contiguous required columns + homogeneous row kernel
- ¶178 **Architecture-to-execution chain:** behavioral-role requirements → traversed tables; access declarations → required columns + concurrency conflicts; no rediscovery from agent-step branches; homogeneous loops → less irrelevant load + vectorization + CPU partitioning; regular arrays + bounded messages/staged kernels → possible GPU batches
- ¶179 **Workload contingencies:** small populations fail to amortize queries; structural changes copy between archetypes; sparse signatures fragment; branch-heavy cognition/tight interaction reduce SIMD/ GPU use; barriers + reductions + conflict resolution dominate; one-individual inspection worsens while population operation clarity improves
- ¶180 **Evaluation implication:** storage performance separate from RQs; fixed transition + varied layout/execution; serial layout vs. thread scaling vs. query/structural/sync/resolution costs; question = conditions of payoff, not guaranteed ECS speed

7 Comparative case study: Sugarscape

7.1 Why Sugarscape

- ¶183 **Canonical model:** heterogeneous citizens + spatial resource landscape → movement + harvest + metabolism + unequal wealth + starvation/ age death; implementation = single-resource toroidal two-hill model + optional sexual reproduction + disease
- ¶184 **RQ fit:** traits → parametric heterogeneity; Female/Male → exclusive nominal roles; Infection attachment/recovery → dynamic structural role; persistent citizen; fertility → sex/age/wealth/neighborhood predicate; clean distinction between component-presence role and derived temporary role
- ¶185 **Mechanism scales:** individual movement + cell/resource competition; neighbor disease; entity-removing lifecycle; parent matching + offspring; environmental growback; sequential/synchronous movement → behavior vs. schedule separation; interpretable outcomes = inequality + mortality + population + prevalence

7.2 Scientific model and extension boundary

- ¶187 **Baseline:** finite torus; fixed capacity + stock growback; cardinal vision; exclude others' occupied cells; maximize sugar → minimize distance → seeded random tie; move + full harvest; metabolism + aging → wealth/lifetime reduction; starvation/age → removal
- ¶188 **Treatments:** default replacement of every death; reproduction and initial infection off; extension allows reproduction + disease; replacement xor reproduction; eligible adjacent female–male + empty neighbor + parental endowment + independently inherited traits; one `UInt64` strain; cardinal transmission + sugar cost + exact-strain immunity + Infection removal; bounded comparison extension, not full original Sugarscape

7.3 Semantics-matched agent-centered reference

- ¶190 **Status/requirement in manuscript:** ECS currently present; idiomatic same-model agent-centered reference required before RQ1/RQ2 conclusions; one mutable citizen record = identity + position + proposal + traits + wealth + age + sex + immunity + optional infection; shared landscape + occupancy + events + clock + RNG; ordinary functions via activation/model coordination; no artificial class weakness
- ¶191 **Matching controls:** common parameters + initial landscape/population + indexed draws + decisions/conflicts + phases + logger; complete world states compared after each phase; only representation + transition-law route differ; prevents semantic confounding

7.4 From agent records to ECS

7.4.1 State composition

- ¶194 **Citizen components:** `CitizenId`, `Position`, `ProposedPosition`, `Vision`, `Metabolism`, `Sugar`, `Age`, `MaximumAge`, `InitialEndowment`, `ImmuneProfile`; exactly one `Female`/`Male`; optional `Infection` = strain + age; resources = landscape + occupancy + RNG + clock + next ID + events + parameters + logger
- ¶195 **Presence as model definition:** Infection addition → progression population; recovery removal → query exit with identity/unrelated state; simultaneous sex + infection + derived fertility/wealth/ mortality without combination type; sex/infection structural vs. fertility/risk computed; explicit stored vs. transient role distinction
- ¶196 **Fair contrast:** optional record field can represent same transition; comparison targets location of schema change + validation + initialization + mutation + logging + selection; no dynamic-composition impossibility claim

7.4.2 Behavior organization

- ¶198 **Period decomposition:** focused population/resource contracts
- ¶199a `growback!` → landscape only
- ¶199b **Movement:** shuffled sequential activation or proposal → resolution → commitment
- ¶199c `disease!` → infectious-neighbor snapshot + staged infection + cost + staged recovery
- ¶199d `lifecycle!` → metabolism + aging + staged deaths + removal + optional replacement
- ¶199e `reproduce!` → compatible-birth planning + cell reservation + parent contribution + offspring creation
- ¶199f `logger!` → population + wealth + resources + movement + mortality + reproduction + disease
- ¶200 **Organization effect:** queries expose participants; resource accesses expose shared inputs; structural commits after traversal; boundaries permit movement/transmission/inheritance substitution under contract; agent-centered helpers may match modularity; RQ2 = natural exposure/reuse, not ECS-only modularity

7.4.3 Schedule semantics

- ¶202 **Declared period order:**
- ¶203 `growback → occupancy → move/harvest → disease → metabolism/age → death/removal/replacement → birth plan/commit → clock/log`
- ¶204 **Scientific dependencies:** growback before choice = replenished observation; movement before transmission = post-move contact; disease before lifecycle = same-period cost/starvation; removal before reproduction = eligibility + empty cells; log after commits = successor state
- ¶205 **Movement treatment:** shuffled sequential → RNG order + immediate vacancy/choice/move/ harvest + later observation of earlier moves/depletion; synchronous → frozen occupancy/landscape + no initially occupied targets + seeded arbitration for shared empty target + loser at origin + post-resolution joint position/wealth/stock/occupancy commit
- ¶206 **Order equivalence only under:** complete conflict-free access sets + stable queries to phase boundary + deterministic resolution + runtime-order- independent draws/reductions; shared RNG currently blocks arbitrary reordering absent indexed/partitioned draws; growback-after-movement, disease-after- lifecycle, reproduction-before-removal = different model, not optimization

7.5 Comparative treatments and evidence

- ¶208 **Design:** common deterministic fixtures + paired-seed experiments
- ¶209a **RQ1 trace:** optional record state vs. tags/dynamic Infection; infection + recovery + birth + death + further capability through definition + initialization + modification + selection + logging
- ¶209b **RQ2 trace:** locality + reuse + substitution + inspectability for movement + transmission + lifecycle + reproduction; population mechanism vs. complete-citizen reconstruction
- ¶209c **RQ3 trace:** sequential vs. synchronous visibility/commit timing; dependency-admissible permutations + complete trajectory equality under indexed randomness + deterministic reductions
- ¶210 **Outcomes:** population + mean/median wealth + Gini + citizen/landscape sugar + movement/ harvest + contests + deaths + replacements + births + prevalence/incidence/recovery; architecture comparison requires prior state/ outcome equivalence; schedule treatment intentionally nonequivalent → timing effects on inequality + access + survival + demography + disease

8 Evaluation design

8.1 Evaluation logic

- ¶228 **Inference constraint:** one attractive code example insufficient; distinct treatment + evidence standard per RQ
- ¶229 **RQ1 matrix:** semantics-matched implementations + roles + infection attachment/recovery; evidence = same-transition representation/modification paths; inference = changing-role representation, not object incapacity
- ¶230 **RQ2 matrix:** same mechanisms via systems/activation + rule substitution; evidence = change locality + reused processes + contracts + tests + traces; inference = locality/reuse/substitution/ population inspectability
- ¶231 **RQ3 matrix:** phases + admissible order + changed visibility/timing; evidence = trajectory equality + outcomes + failed-condition counterexamples; inference = schedule semantics + preservation conditions

8.2 Experiment A: State composition

- ¶233a **Control:** identical state variables + mechanisms + schedule + parameters + seeded draws across idiomatic agent-centered/ECS
- ¶233b **Fixture:** overlapping sex + fertility + infection; attachment/ removal at phase boundaries
- ¶233c **Extension test:** one new capability after both implementations
- ¶233d **Comparison path:** presence + role change through state definitions + validation + initialization + transition + logging + analysis
- ¶233e **Evidence:** qualitative dependency changes + limited touched modules + duplicated logic + represented combinations
- ¶234 **Conclusion boundary:** locality/composability, not impossibility in objects

8.3 Experiment B: Behavior organization and substitution

- ¶236a **Control:** scientific state + schedule + entities + unrelated mechanisms fixed
- ¶236b **Locality:** changed code + declared dependencies for one population mechanism
- ¶236c **Reuse:** unchanged mechanism code across capability profiles/ treatments
- ¶236d **Substitutability:** movement/transmission/reproduction/conflict rule swap under common I/O contract + boundary tests
- ¶236e **Inspectability:** participants + inputs + outputs + schedule position; separate whole-agent reconstruction
- ¶236f **Scientific variants:** movement mode + reproduction + disease enabled by mechanism separation

8.4 Experiment C: Schedule semantics

- ¶238 **Comparison 1:** sequential vs. synchronous → visibility + depletion + conflict + commitment consequences; **comparison 2:** fixed phases + alternative topological system orders; preservation preconditions = complete access sets + conflict freedom/commutative reduction + stable membership + schedule- independent draws + deterministic resolution + fixed floating reduction; complete trajectories + deliberate violations
- ¶239 **Changed-semantics controls:** behavioral equations + initial conditions fixed; measured outcomes
- ¶240a **Wealth:** mean + median + Gini
- ¶240b **Resources:** citizen sugar + landscape sugar + harvest
- ¶240c **Movement/mortality:** movements + contested destinations + death causes
- ¶240d **Demography/disease:** population + births + replacements + prevalence + recoveries
- ¶240e **Sensitivity:** density + regeneration + reproduction + disease
- ¶241 **Two claims separated:** changed visibility/timing may change outcomes; independence-satisfying alternative orders should preserve outcomes

8.5 Secondary engineering evaluation

- ¶243 **Benchmark set:**
- ¶244a agent-centered serial
- ¶244b ECS serial
- ¶244c ECS threaded
- ¶244d ECS GPU only if complete/fair
- ¶245 **Reported dimensions:**

- ¶246a warmed post-compilation wall time
- ¶246b population/component scaling
- ¶246c allocations + peak memory
- ¶246d cache misses + bandwidth where measurable
- ¶246e thread scaling + parallel efficiency
- ¶246f query/system/structural/synchronization/conflict time
- ¶246g hardware + software + compiler + repetitions
- ¶247 **Workloads:** compute-light + interaction-heavy; synthetic sweep for layout isolation; end-to-end Sugarscape baseline + reproduction + disease

8.6 Randomness, inference, reproducibility

- ¶249a **Randomness:** pregenerated/indexed by replicate + time + entity + event; no architecture-specific stream-consumption artifact
- ¶249b **Pairing distinction:** matched initialization vs. truly paired later shocks
- ¶249c **Analysis plan:** primary contrasts/outcomes predefined
- ¶249d **Inference:** Monte Carlo uncertainty + effect sizes
- ¶249e **Robustness:** longer horizons + multiple initial conditions
- ¶249f **Archive:** code + manifests + raw replicate data + exact commands

9 Results — required structure, not current findings

- ¶251 **Status:** no prose findings yet; organize by RQ, not script order; each subsection = one-sentence answer → evidence + uncertainty

9.1 RQ1: State composition

- ¶253a component-signature count + distribution
- ¶253b same overlapping roles + capability addition/removal in both forms
- ¶253c locality/dependency evidence; code size ≠ scientific proof
- ¶253d architecture effects separated from changed capability distribution

9.2 RQ2: Behavior organization

- ¶255a system dependency graph
- ¶255b reused systems across behavioral variants
- ¶255c unit/integration tests at system boundaries
- ¶255d locality + reuse + substitution vs. matched agent-centered reference
- ¶255e population-process inspection vs. complete-agent inspection

9.3 RQ3: Schedule semantics

- ¶257a schedules with intentionally different information/update timing
- ¶257b fixed phases + admissible order → trajectory equality
- ¶257c each equivalence claim → dependency + stable query + randomness + resolution + reduction conditions
- ¶257d removed dependency/phase boundary → outcome-change counterexamples

9.4 Secondary engineering results

- ¶259a layout gains separated from threading gains
- ¶259b scaling curves, not one population size
- ¶259c ECS-overhead crossover points
- ¶259d little/no-advantage workloads

10 Discussion

10.1 Joint meaning of the three answers

- ¶262 **Synthesis requirement:** evidence, not architecture restatement
- ¶263a **RQ1:** signatures → overlapping/changing-role representation + modification vs. idiomatic agent-centered same-model implementation
- ¶263b **RQ2:** system boundaries → locality + reuse + substitution + inspectability of population mechanisms

- ¶263c **RQ3:** dependencies/phases → visibility + timing + declared outcome-preserving order conditions
- ¶264 **Engineering placement:** whether same component/dependency information improves tested execution; not fourth research answer
- ¶265 **Agency answer:** agent = identity + private state + information + feasible actions + consequences; object = one implementation; ECS externalizes shared transition laws without eliminating entity state/ autonomy

10.2 Where alignment breaks down

- ¶267a no scientifically correct timing semantics supplied
- ¶267b no automatic parallelization of conflicting interactions
- ¶267c no cache/GPU performance guarantee
- ¶267d no replacement for validation + calibration + sensitivity
- ¶267e possible loss of whole-agent behavior inspectability
- ¶267f possible excess for small + single-stable-type + individualized models

10.3 Implications for economic ABMs

- ¶269 **Beyond-Sugarscape examples required:**
- ¶270a **Firm:** exporter + borrower + employer + innovator capabilities; combinations without nominal-type explosion
- ¶270b **Household:** labor + credit + housing + consumption participation by current components
- ¶270c **Institutions:** system/resource vs. agent representation → explicit substantive agency choice
- ¶270d **Dynamics:** entry + bankruptcy + learning + institutional change as composition transformations
- ¶271 **Evidence boundary:** implications until implemented case studies

10.4 Threats to validity

- ¶273a one Sugarscape model ≠ universal architectural superiority
- ¶273b agent-centered implementation potentially biased by author familiarity or framework constraints
- ¶273c chosen library potentially conflates abstract ECS with storage
- ¶273d performance hardware/workload dependence
- ¶273e dynamic composition → timing/visibility choices
- ¶273f explicit entity/agent/environment distinction required

11 Conclusion

- ¶275 **Position:** ECS in ABM = alternative scientific specification architecture, not performance technique only; main promise = changing capability composition + explicit population processes + simultaneous-interaction dependencies; cache/parallel benefits important but empirical + workload-dependent
- ¶276 **Next research:** larger economic model + multiple institutional roles + endogenous capability changes

A Complete ODD description

A.1 Purpose, entities, environment, scale

- ¶212 **Purpose:** local movement/harvest on regenerating unequal landscape → wealth distribution; reproduction/disease modifications; citizens = agents; sugar patches = toroidal environment; full declared schedule = one period

A.2 State and initialization

- ¶214 **Citizen state:** identity + position + proposal + vision + metabolism + sugar + age + maximum age + initial endowment + sex + immune profile; optional Infection; seeded distinct-cell placement; deterministic two-hill capacities unless valid supplied matrix
- ¶215 **Defaults:** 50×50; 400 citizens; capacity 4; growback 1; vision 1–6; metabolism 1–4; sugar 5–25; lifespan 60–100; death replacement; reproduction/infection disabled

A.3 Process overview

- ¶217 **Period:** grow sugar → occupancy → selected movement schedule → disease → metabolism/age → death removal → configured replacement/offspring → clock → aggregates; all randomness from seeded model RNG; structural changes staged between queries

A.4 Movement and harvesting

- ¶219 **Choice set:** current + cardinal visible torus cells; exclude others' occupancy; objective hierarchy = sugar max → distance min → random tie; destination stock to zero + wealth increment
- ¶220 **Sequential:** occupancy/stock updated after each citizen; **synchronous:** frozen proposal state + grouped destinations + one contested-cell winner + winners/nonmoving losers committed + occupancy rebuilt

A.5 Disease, lifecycle, regeneration

- ¶222 **Disease:** cardinal post-movement neighbors; susceptible + no exact immunity → one neighbor strain; infection age + sugar cost same period; duration reached → stored strain immunity + Infection removal
- ¶223 **Lifecycle:** metabolism subtraction + age increment; nonpositive sugar → starvation; age beyond max → old-age death; replacement only in replacement regime; reproduction regime → eligible adjacent female/male + reserved empty neighbor + half initial-endowment contributions + independently inherited vision/metabolism/max-age + sex tag

A.6 Recorded outcomes

- ¶225 **Logger:** population + mean/median wealth + Gini + mean age + citizen/ landscape sugar + movements + conflicts + harvest + cause-specific deaths + replacements + births + prevalence + incidence + recoveries; complete citizen snapshots → deterministic architecture/schedule state comparison

B Supplementary architecture comparison

- ¶281a full agent-centered pseudocode
- ¶281b full ECS table = queries + reads + writes + structural changes
- ¶281c dependency graph + execution phases

C Supplementary experimental design

- ¶283a infection-role + reproduction treatments
- ¶283b density + resource + horizon + disease grids
- ¶283c replicate counts + seed construction
- ¶283d primary + secondary outcomes

D Additional results

- ¶285a full movement-schedule comparison
- ¶285b reproduction + disease sensitivity
- ¶285c longer-horizon + landscape + initialization robustness
- ¶285d admissible-order state equivalence
- ¶285e complete performance profiles

E Reproducibility

- ¶287a repository + archived release
- ¶287b Julia + Rust + Typst + package versions
- ¶287c hardware + operating system
- ¶287d test + simulation + figure + benchmark + paper commands